Reject-Before-Propagate: Build-Time Citation Gates for Agentic Literature-Synthesis Pipelines

N. Harish Vidyarth*, A. L. Reshma, R. Divya
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College, Thandalam, Chennai 602105, India
*Corresponding author: harishvidyarth.n.2024.csecs@rajalakshmi.edu.in

***Abstract*—Large language model (LLM) pipelines for automated literature review chain several generative stages whose output is consumed downstream and, ultimately, by people making real decisions. Current systems treat citation grounding as a *scored* property: a statement is checked against retrieved evidence and assigned a confidence value, flagged, or queued for review, but is not structurally prevented from reaching the final output when that check fails. We present *Verify-or-Reject Grounding* (VoRG), an architectural pattern that treats citation grounding as a build-time invariant: each stage either produces output whose type excludes unverifiable content, or raises a build error. We give a formal statement of the pattern, VoRG-Certify, an $O(n+M)$ narrative-certification algorithm that removes the $O(n{\cdot}m)$ term of a naive per-sentence scan, and a reference implementation across three subsystems of an eight-stage pipeline. On a reproducible gold-annotated pilot, the two verify-or-reject-gated stages produce no output contradicted by the reviewed corpus, while research-gap detection, the one unprotected stage, produces a confirmed false claim that a corpus paper directly contradicts. This gap-to-failure correspondence, between architectural coverage and failure location, is the paper's central finding.**

***Index Terms*—verify-or-reject grounding, literature synthesis, large language models, citation grounding, hallucination mitigation, agentic NLP**

I. Introduction

LLMs are increasingly used to accelerate literature review by extracting claims, identifying disagreements, and drafting synthesis prose. A documented failure mode is *hallucination*: a statement appears well supported but matches nothing in the source [1], [2]. A related failure is citation-level: the cited source may not say what is claimed, or may not exist [3]. Standard mitigations rely on post-hoc verification, attaching a score, a flag, or a review-queue entry to the output [4]. Because validation is appended to the output rather than made a gate on it, an unverified or contradictory statement still reaches the user if the score is not acted on. When the output feeds a real submission, a hallucinated citation is costly.

We take a different design stance. A stage is *verify-or-reject grounded* if its output type itself excludes unverifiable content, either because the stage is deterministic over already-verified inputs, or because an explicit check discards non-conforming output before it can propagate. Scored grounding is a property of the *output*, inspectable but skippable; verify-or-reject grounding is a property of the *pipeline*, which a downstream stage cannot bypass by ignoring a score it was never given. This is the reject-by-default logic of the fail-safe-defaults principle in protection-system design [5], applied to citation grounding instead of access control.

Contributions:

- A formal statement of VoRG as a reusable architectural pattern, and VoRG-Certify, an $O(n+M)$ certification algorithm that removes the $O(n{\cdot}m)$ term implicit in a naive per-sentence evidence scan.
- A reference implementation applying the pattern to three subsystems of an eight-stage research-assistant pipeline, released with its evaluation harness.
- A reproducible evaluation reporting a *gap-to-failure correspondence*: the gated stages produce no corpus-contradicted output, whereas the single ungated stage produces a false claim that a corpus paper contradicts.

II. Related Work

AI-assisted review and agentic LLMs. Li *et al.* [4] build a GPT-4 systematic-review system with a human-in-the-loop safeguard; a low-confidence decision is surfaced to the reviewer, not withheld. Tools such as LLAssist [6] automate relevance screening and information extraction with an LLM scoring each paper's fit to a research question. Surveys of LLM-automated review find that most systems automate a single stage with a confidence score or a human check rather than a chained, structurally gated pipeline [7], [8]. Retrieval-augmented generation [9] grounds generation in retrieved passages but does not reject ungrounded output; ReAct-style agents [10] lack a citation-level accept/reject rule; Self-RAG [11] learns a reflection signal but keeps it soft rather than a hard external gate. Stage-level evaluation follows agentic-LLM benchmarks such as AgentBench [12].

Contradiction detection. Surveys of contradiction detection in biomedical literature [13] catalogue linguistic and sta-

tistical approaches with no explicit reject path: a contradiction is asserted or not, with no abstention grounded in evidence availability. Natural-language inference [14] treats contradiction as a three-way classification rather than a judgment that can be withheld when supporting evidence is thin, and using an LLM as the judge adds a reliability question [15] that a bare classification score does not surface.

Citation faithfulness. Hallucination-mitigation surveys and benchmarks such as TruthfulQA [16], HaluEval [17], and SelfCheckGPT [18] quantify or detect unsupported content *post hoc*. Closer to citation grounding, attribution frameworks [19], [20] and benchmarks such as ALCE [21] and CiteME [22] *measure* whether a generated statement is attributable to a source; ALCE finds strong LLMs leave citation claims unsupported roughly half the time, and CiteME finds frontier models identify a cited paper far below the roughly 70% accuracy of humans. The evidence-retrieval step of fact-verification tasks such as FEVER [23] is structurally close to our extraction gate, except an unsupported extraction here is discarded, not scored "not enough info". Reporting-transparency checklists such as PRISMA-trAIce [24] standardise what authors *report* about AI use, whereas VoRG constrains what the *system* is permitted to emit. Bibliometric sources also disagree on journal quality and coverage [25], [26], motivating explicit reconciliation rather than a single trusted database. All of the above report how faithful an output is; VoRG prevents an unattributable statement from being emitted at all.

III. System Architecture

The reference system is a pipeline with eight sequential stages. *Topic expansion* decomposes the input into subtopics, methods, datasets, and keywords, with an explicit instruction not to invent a dataset name; when no provider is configured it falls back to a keyword dictionary, which is also the rule-based configuration used for the pattern-mode ablation. *Multi-source discovery* queries arXiv, Semantic Scholar, and CrossRef concurrently and deduplicates by DOI, normalised title, and fuzzy title similarity within a one-year window. *Claim normalisation* folds metric-name synonyms and units into a common vocabulary and applies a rule-based confidence adjustment from hedging and strength language; this is deliberately a scored, not gated, step, sitting directly upstream of the gated contradiction stage. *Citation building* is a deterministic transformation of already-verified metadata. *Persistence* keeps run state as JSON and journal-quality data in a relational schema, one table per external source, joined on ISSN, each row carrying its own snapshot date. Three subsystems enforce verify-or-reject grounding explicitly; the rest are ordinary generation or aggregation steps that feed a gate.

- **Evidence-grounded synthesis** (Fig. 1). Every extracted field must be a verbatim or near-verbatim substring of the source before entering an evidence table. A synthesis-matrix stage aggregates the table into a methods-by-datasets grid deterministically, with no LLM call. A narrative verifier rejects any generated sentence whose citation key does not resolve to a verified row; an unsupported sentence is a build error.
- **Contradiction detection.** Claim pairs are pre-filtered by TF-IDF similarity, then judged by an LLM on an 11-point Likert scale. Judgments below a threshold are discarded, not reported with low confidence.
- **Journal-metadata reconciliation.** A metric is reported only when a source explicitly provides it; cross-source disagreements above 10% are surfaced in a discrepancy field rather than averaged.

TABLE I
Pipeline stages and their grounding discipline.

Stage	Grounding	Gated
Topic expansion	prompt-level instruction	no
Multi-source discovery	deduplication only	no
Claim extraction	feeds a gate	no
Claim normalisation	scored confidence adjustment	no
Contradiction detection	discard below threshold	**yes**
Research-gap detection	none	no
Citation building	deterministic transform	yes
Persistence	never-merge-silently	yes
Evidence-grounded synthesis	verbatim check, verifier	**yes**

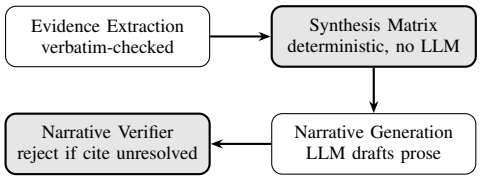


Fig. 1. Evidence-grounded synthesis sub-pipeline. Shaded boxes are hard gates: output that cannot be verified is not produced (deterministic aggregation) or is discarded (build-error rejection), never passed through with a soft score.

A. Implementation

The system is implemented in Python with a Flask interface. Each evidence row carries a fixed schema (population, method, dataset, metric, result, limitation) plus a verbatim-evidence field that is populated only when the model's supporting sentence is a normalised substring of the source abstract; a row that fails this check in reject mode is replaced by a deterministic rule extraction rather than admitted at low confidence. LLM calls use Groq with the model `openai/gpt-oss-120b`. The evaluation harness has three entry points: `snapshot_topic` freezes a corpus and backfills missing abstracts from Semantic Scholar and OpenAlex before recording per-corpus coverage; `run_eval` runs the frozen corpus in one of four configurations and emits per-run predictions, per-topic metrics, and the downstream-error block; and `run_matrix` sweeps every topic-by-configuration-by-repeat cell and aggregates per-topic mean and standard deviation with the repository commit hash recorded in the output. Predicted claims are matched to gold by paper identifier plus a normalised fuzzy-ratio threshold; contradictions are matched at the unordered paper-pair level, with a hit on a gold-negative pair counted as an explicit false positive; gaps are matched

when the type agrees and either the description similarity or the keyword overlap exceeds a threshold.

IV. Verify-or-Reject Grounding

A. Definitions

A stage is *scored-grounded* if it emits output alongside a confidence signal that a downstream consumer may act on or ignore. A stage is *verify-or-reject grounded* if its output type excludes unverifiable content, either because it is deterministic over verified inputs (no new hallucination surface) or because an explicit step discards non-conforming output before it propagates. Table II contrasts the two.

The design rationale is that a score is a value a later stage is free to drop, whereas a type is a contract it cannot. Placing the check inside the stage's output type moves the guarantee from "the consumer should inspect this" to "the consumer cannot receive an unverified value". This is why the pattern is stated as a property of the pipeline rather than of any single output: a downstream stage added later inherits the guarantee without having to know the score field existed. The cost is that a stage which cannot verify its output must fail loudly rather than degrade quietly, which is acceptable when the output feeds a document a person will sign.

B. Certification Algorithm

A naive scored-grounding implementation would check each generated sentence against the evidence table by linear scan. VoRG-Certify instead compiles the evidence table into a hash index keyed by citation key, then certifies the whole narrative in a single pass (Algorithm 1), so certification cost scales with the size of the narrative and the evidence table separately rather than with their product. A sentence with no citation key, or a key that does not resolve, is logged and dropped rather than emitted; under verify-or-reject the second case is a build error.

C. Complexity

Let $n = |E|$, m the number of generated sentences, and $M = \sum_j |s_j|$ the total narrative length. BuildIndex is $O(n)$ time and space. Certify makes one pass: key extraction is $O(M)$ across all sentences and each lookup is expected $O(1)$, so

$$T(n, M) = O(n + M), \qquad S(n, m) = O(n + m).$$

A naive scored implementation checks every sentence against every evidence row, costing $O(n \cdot m)$; the hash-indexed design removes this quadratic term, so certification stays linear as the evidence base scales. The same index-then-certify structure applies to contradiction judgments (keyed by claim pair) and journal-metadata fields (keyed by source), so the bound extends to those subsystems.

Algorithm 1 VoRG-Certify

Require: evidence table $E = \{e_1, \ldots, e_n\}$, each row keyed by $\kappa(e_i)$; narrative $N = (s_1, \ldots, s_m)$
Ensure: certified narrative N'; rejection log R

function BuildIndex(E)
  $H \leftarrow \emptyset$
  for each $e_i \in E$ **do** $H[\kappa(e_i)] \leftarrow e_i$ {expected $O(1)$}
  return H
function Certify(N, H)
  $N' \leftarrow \emptyset$; $R \leftarrow \emptyset$
for each sentence $s_j \in N$ **do**
  $K_j \leftarrow$ ExtractKeys(s_j) {one regex scan, $O(|s_j|)$}
  if $K_j = \emptyset$ **then**
    $R \leftarrow R \cup \{(s_j, \text{NoClaim})\}$
  else if $\exists\, k \in K_j :\ k \notin H$ **then**
    $R \leftarrow R \cup \{(s_j, \text{Unresolved})\}$ {build error}
  else
    $N' \leftarrow N' \cup \{s_j\}$
  end if
end for
return (N', R)

V. Evaluation

RQ1. Does replacing each gate with a matched confidence score, with the stage decomposition fixed, change the rate at which unsupported or contradicted output reaches the final synthesis? **RQ2.** Does the ungated stage (research-gap detection) fail in a way the gated stages do not, and does the failure location match the architectural coverage? **RQ3.** How much of any claim-extraction gain is from stage decomposition rather than from the LLM calls or the rejection mechanism?

A. Setup

Each topic is snapshotted once and frozen, so scored runs issue no paper-source traffic; missing abstracts are backfilled from Semantic Scholar and OpenAlex and per-corpus abstract coverage is reported. Gold labels are authored by independently correcting pipeline output, not by rubber-stamping it. Four configurations are compared, each repeated three times on the pilot (temperature 0.3, Groq `openai/gpt-oss-120b` where an LLM call is used); the four-topic extension (Section V-D) uses one repeat per configuration on a local model (Ollama `qwen2.5:7b`): (i) the full pipeline with all gates on; (ii) a *matched scored-grounding ablation* with the same decomposition and LLM calls but every gate replaced by a score, nothing dropped; (iii) a pattern-mode ablation with rule-based extraction; (iv) a naive single-prompt baseline. Downstream-error metrics, reported for every configuration: unsupported sentences in the final narrative (zero by construction under verify-or-reject), retained false-positive contradictions, and unflagged metric conflicts.

B. Pilot Result

The pilot topic is *graph neural networks in drug discovery*, a frozen 10-paper CrossRef corpus. In the pilot snapshot only

TABLE II
SCORED GROUNDING VERSUS VERIFY-OR-REJECT GROUNDING.

	Scored grounding	**Verify-or-reject grounding**
Unverifiable output	emitted with a low score/flag	not emitted; build error raised
Downstream dependence	consumer must check the score	consumer receives only verified output
Aggregation steps	often LLM-driven, a hallucination source	deterministic where possible, no new LLM surface
Failure visibility	silent unless the score is inspected	explicit, logged rejection

TABLE III
CLAIM-EXTRACTION PERFORMANCE ON THE PILOT TOPIC (MEAN ± SD, 3 RUNS, 3 GOLD CLAIMS). UNDERPOWERED AT THIS GOLD-SET SIZE; NOT USED TO COMPARE EXTRACTOR TYPES.

Configuration	P	R	F1
Full pipeline (verify-or-reject)	0.294	0.889	0.442
Pattern-mode ablation	0.439	0.667	0.510
Naive single-LLM baseline	0.104	0.333	0.158

TABLE IV
UNSUPPORTED SENTENCES REACHING THE FINAL SYNTHESIS, VERIFY-OR-REJECT VERSUS THE MATCHED SCORED ABLATION, ONE RUN PER TOPIC. ON THREE OF FOUR TOPICS VERIFY-OR-REJECT ENFORCES ZERO LEAKED SENTENCES BY *failing the build outright* RATHER THAN BY PASSING EVERY SENTENCE; SCORED MODE ALWAYS COMPLETES AND ITS LEAK COUNT IS DIRECTLY OBSERVABLE.

Topic	Reject (verify-or-reject)	Scored: unsup. / total
GNN drug discovery (pilot)	0 / 4	1 / 5
DL malware detection	build-failed	8 / 24
GNN fraud detection	build-failed	29 / 76
Net. anomaly detection	build-failed	17 / 66
pooled (new topics)	**0 reach reader**	**54 / 166**

one paper carried a non-empty abstract, limiting the verifiable claim gold set to three sentences; the abstract-backfill step was added in response and applies to later topics.

Both the full pipeline and the pattern-mode ablation beat the naive baseline (F1 $2.8\times$ and $3.2\times$), so decomposing extraction into per-paper, per-claim stages helps regardless of extractor (RQ3). The pattern-mode point estimate is nominally higher, but its precision variance (± 0.211) is five times the full-pipeline's (± 0.042); with three near-verbatim gold sentences a regex extractor tuned to sentence boundaries is mechanically favoured. The multi-topic study is designed to settle this.

Contradiction detection. Across all nine runs the false-positive rate on the two hand-labelled negative pairs was 0.0. No gold-positive contradictions exist in this corpus, so recall is undefined; this is a precision-only sanity check.

Research-gap detection (negative result). Gap recall against two gold gaps was 0.0 in two of three full-pipeline runs and 0.5 in the third (mean 0.17); precision@3 was 0.0 in every run. One run's top-ranked gap claimed that "no studies have applied graph-based generative models ... for de novo molecular generation", directly contradicted by the corpus's own sixth paper, "Molecular generative Graph Neural Networks for Drug Discovery". Gap detection is the one component that does *not* apply verify-or-reject grounding: a candidate gap is generated and ranked but never checked against the corpus it judges.

C. Gap-to-Failure Correspondence

The gated and ungated stages diverge as the architecture predicts. Claim extraction and contradiction detection pass every candidate output through an explicit accept/reject check, and neither produced output contradicted by the corpus. Research-gap detection has no such check, and it is precisely there that the pipeline produced a false claim contradicted by a paper already in the corpus. This correspondence emerged during evaluation of the existing pipeline; its location matches the one architectural gap in Section IV. We treat this, rather than the F1 numbers, as the strongest evidence for build-time rejection over post-hoc scoring. The journal-metadata subsystem shows the same discipline on structured output: values are tagged verified, estimated, or unknown, and source conflicts are surfaced rather than averaged.

D. Multi-Topic Study (preliminary)

We extend the pilot to four topics: the adjudicated *graph neural networks in drug discovery* plus *deep learning for malware detection*, *anomaly detection in network traffic*, and *graph neural networks for fraud detection*. The three added topics use *bootstrapped candidate gold* — pipeline output promoted without independent adjudication — so for them we report only gold-independent measurements: the downstream-error counts (Table IV) and the gap-to-failure correspondence (Table V). Claim P/R/F1 against adjudicated gold remains a pilot-only result (Table III). Each configuration is run once (one repeat) on a local model, `Ollama qwen2.5:7b`; the two larger corpora were trimmed to eight papers each to bound wall-clock time.

On three of the four topics, reject mode's "zero unsupported sentences" invariant is enforced by refusing to finish the paragraph at all: the LLM reliably writes one uncited framing sentence per synthesized paragraph (for example, "The application of advanced graph neural networks in fraud detection has shown promising results."), and the verifier's requirement that every sentence carry a `\cite{}` key has no tolerance for it. We strengthened the correction-retry prompt to name the citation requirement explicitly and re-tested on a larger model (`qwen2.5:14b`); neither change altered the outcome, so we report the build failures as a genuine result rather than continue tuning around them. Scored mode, facing the identical synthesis task, completes every time and lets a

TABLE V
GAP-TO-FAILURE CORRESPONDENCE PER TOPIC: RESEARCH GAPS FROM THE *ungated* STAGE THAT A PAPER IN THE SAME CORPUS DIRECTLY CONTRADICTS (HAND-VERIFIED AGAINST CORPUS TITLES AND ABSTRACTS). THE GATED STAGES PRODUCED NONE IN ANY TOPIC.

Topic	Gaps	Corpus-contradicted	Gated
GNN drug discovery (pilot)	2	1	0
DL malware detection	3	2	0
GNN fraud detection	4	3	0
Net. anomaly detection	5	3	0

substantial fraction of its sentences through unverified (54 of 166 pooled, 33%) — the two modes' behavior under the same failure-inducing input is itself the finding: reject mode trades completion for the zero-leak guarantee, scored mode trades the guarantee for completion.

Across four topics the ungated research-gap stage produced 9 gaps that a paper already in the same corpus directly contradicts — for example, in fraud detection a gap claiming "no work on spectral graph networks" against *Learning frequency-aware graph fraud detection*, and in network-anomaly detection a gap claiming no deep-learning work exists against *FGAN: Federated Generative Adversarial Networks for Anomaly Detection in Network Traffic* — while the two verify-or-reject-gated stages produced none, on every topic measured. The correspondence between the one unprotected stage and the one location of corpus-contradicted output, first seen on the pilot, holds on all four topics.

E. Discussion

Nothing about the pattern is specific to literature review. Any pipeline that generates text or structured records from retrieved sources can apply the same accept/reject check before passing output downstream, in place of a confidence score a later stage might not act on. The results do not show that VoRG solves hallucination in every setting; what the evaluation establishes is where verification is already load bearing and where its absence produces a concrete, corpus-contradicted failure. Extending the gate to research-gap detection, and testing on larger and more varied corpora, are the most direct next steps.

F. Reproducibility

Every results directory records the repository commit hash, the provider, the model, the temperature, per-topic abstract coverage, per-run cost, and parse-failure counts. All scored runs read the frozen corpus snapshots, so re-running the evaluation issues no traffic to paper sources and depends only on the pinned model endpoint. The frozen corpora, the gold-annotation files, the harness, and the matrix driver are released with the reference implementation, so a reader can reproduce every number in this section and extend the study to new topics by adding a snapshot and its gold files.

VI. THREATS TO VALIDITY

Construct. The matched scored-grounding ablation operationalises "the same pipeline without the gate" by fixing decomposition and LLM calls and varying only accept/reject versus score; the naive baseline tests decomposition, not scoring. *Internal.* Two pre-existing implementation defects (a retired model identifier; a schema-validation issue rejecting valid null fields) were fixed during evaluation; reported results reflect the corrected system. Providers are non-deterministic even at low temperature; we report mean and SD over three runs and pin the provider, model, and temperature. *Statistical.* The pilot claim gold set has three sentences; its variance is indicative only and its F1 ordering is not used to draw extractor-type conclusions. *Reliability.* On three of four multi-topic runs, verify-or-reject's zero-leak guarantee manifested as an outright build failure rather than a clean zero, because the synthesis model reliably emits one uncited sentence per paragraph regardless of model size (7B or 14B) or a correction-retry that names the citation requirement explicitly. We report this as a finding, not a harness defect papered over: a build-time gate that refuses incomplete output is doing exactly what it is specified to do, at the cost of availability. *External.* The pilot covers one topic and one annotator team; results are a case study. The four-topic extension adds breadth on gold-independent measurements only, since three of its topics use bootstrapped, unadjudicated gold.

VII. CONCLUSION

We described Verify-or-Reject Grounding, an architectural pattern that treats citation grounding as a build-time invariant, gave a linear-time certification algorithm, and implemented it across three subsystems of an eight-stage pipeline. The evaluation's most robust result is a correspondence between architectural coverage and failure location: the gated stages produced no corpus-contradicted output, while the one unprotected stage produced a confirmed false claim. Future work will (i) apply the gate to research-gap detection, (ii) run the matched scored-grounding ablation and broaden to three to five gold-annotated topics, and (iii) test whether the correspondence holds across scientific domains. The reference implementation, evaluation harness, and gold files are released at https://github.com/harishvidyarthcsecs/Research.

REFERENCES

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, p. 248, 2023.

[2] W. Zhang and J. Zhang, "Hallucination mitigation for retrieval-augmented large language models: A review," *Mathematics*, vol. 13, no. 5, p. 856, 2025, cited as general background; journal quartile not independently confirmed at time of writing.

[3] M. Zhang and T. Zhao, "Citation accuracy challenges posed by large language models," *JMIR Medical Education*, vol. 11, p. e72998, 2025.

[4] Y. Li, S. Datta, M. Rastegar-Mojarad, K. Lee, H. Paek, J. Glasgow, C. Liston, L. He, X. Wang, and Y. Xu, "Enhancing systematic literature reviews with generative artificial intelligence: development, applications, and performance evaluation," *Journal of the American Medical Informatics Association*, vol. 32, no. 4, pp. 616–625, 2025.

[5] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[6] C. Y. Haryanto, "Llassist: Simple tools for automating literature review using large language models," 2024, presented at CIE51, 2024.

[7] D. Scherbakov, N. Hubig, V. Jansari, A. Bakumenko, and L. A. Lenert, "The emergence of large language models as tools in literature reviews: A large language model-assisted systematic review," *Journal of the American Medical Informatics Association*, vol. 32, no. 6, p. 1071, 2025.

[8] R. van Dinter, B. Tekinerdogan, and C. Catal, "Automation of systematic literature reviews: A systematic literature review," *Information and Software Technology*, vol. 136, p. 106589, 2021.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," 2023, iCLR 2023.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," 2024, iCLR 2024.

[12] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "Agentbench: Evaluating llms as agents," 2024, iCLR 2024.

[13] A. Alamri and M. Stevenson, "A survey of technologies for detecting contradiction in biomedical literature," *Journal of Biomedical Semantics*, vol. 7, p. 36, 2016.

[14] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018, pp. 1112–1122.

[15] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[16] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 3214–3252.

[17] J. Li, X. Cheng, X. Zhao, J.-Y. Nie, and J.-R. Wen, "Halueval: A large-scale hallucination evaluation benchmark for large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[18] P. Manakul, A. Liusie, and M. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017.

[19] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023.

[20] B. Bohnet, V. Q. Tran, P. Verga, R. Aharoni, D. Andor, L. B. Soares, M. Ciaramita, J. Eisenstein, K. Ganchev, J. Herzig, K. Hui, T. Kwiatkowski, J. Ma, J. Ni, L. S. Saralegui, T. Schuster, W. W. Cohen, M. Collins, D. Das, D. Metzler, S. Petrov, and K. Webster, "Attributed question answering: Evaluation and modeling for attributed large language models," 2022.

[21] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[22] O. Press, A. Hochlehnert, A. Prabhu, V. Udandarao, O. Press, and M. Bethge, "Citeme: Can language models accurately cite scientific claims?" in *Advances in Neural Information Processing Systems 38 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024.

[23] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: a large-scale dataset for fact extraction and VERification," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018, pp. 809–819.

[24] JMIR AI Editorial, "Transparent reporting of ai in systematic literature reviews: Development of the prisma-traice checklist," *JMIR AI*, vol. 4, p. e80247, 2025, cited as general background; journal quartile not independently confirmed at time of writing.

[25] V. Macháček and M. Srholec, "Predatory publishing in scopus: Evidence on cross-country differences," *Quantitative Science Studies*, vol. 3, no. 3, pp. 859–887, 2022.

[26] P. Mongeon and A. Paul-Hus, "The journal coverage of web of science and scopus: A comparative analysis," *Scientometrics*, vol. 106, no. 1, pp. 213–228, 2016.